

Instructor-Produced Multimedia Learning Objects in Computer Science Education

Sabbatical Report
Fall 2007 & Spring 2009

Professor Stephen Gilbert
Orange Coast College
Computer Science Department
August 2009

Original Sabbatical Proposal.....	2
Sabbatical Leave Report.....	7

<http://faculty.orangecoastcollege.edu/sgilbert/SabbaticalReport.pdf>

September 11, 2006

To: Sabbatical Leave Committee
Orange Coast College

From: Stephen Gilbert
Associate Professor Computer Science
Computer Science Department
Orange Coast College

Re: Sabbatical Leave Proposal

Proposed dates of Sabbatical Leave: Fall 2007 and Spring 2009 (Split)

Type of leave requested: Independent Study/Research

Title: Study of C# and Microsoft .NET Programming and its application to undergraduate Computer Science education using instructor-produced multimedia learning objects

Goals: The goals of this sabbatical are to develop new skills in producing multimedia learning objects, and to apply those skills to a pilot project in our department's Microsoft .NET Programming offerings.

Current Teaching Assignments:

These are the classes I normally teach each year:

- CS 111 – Introduction to Information System and Programming (3 sections)
- CS 170 – Java Programming I (4 sections, 2 online)
- CS 150 – C++ Programming I (1 section in Spring)
- CS 140 – C# .NET Programming (2 sections 50% online)
- CIS 223 – Advanced Web Page Design (1 section in Spring, 100% online)

These are the classes I teach as need arises:

- CS 250 – C++ Programming II (When Shin K. is gone in Fall)
- CS 200 – Data Structures
- CS 240 – Microcomputer System Development (Assembly Programming)
- CS 272 – Java Programming II (Offered when my schedule permits)

Narrative:

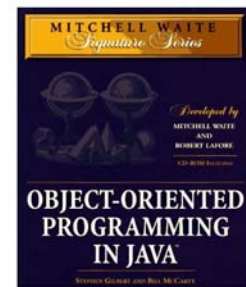
I started teaching rather late in my life. My first year of full-time teaching was 1994 when I was hired at Azusa Pacific University to teach undergraduate and graduate Computer Science classes. The year before I started full time at APU, the department has switched their undergraduate teaching language from Pascal to C++. Industry was moving away from procedural language and towards object-oriented languages, and many in the department thought that Pascal, which was designed as a teaching language, had outlived its usefulness.

One of the reasons that I was hired was because I knew C++ fairly well, and the administration thought it would be easier for me to teach the introductory freshmen and sophomore classes, than to retrain the instructors who had been teaching Pascal for the last

ten years. This "experiment", (which included about 30% of the Computer Science departments across the country at the time), wasn't a complete failure, but it wasn't an unqualified success either. While C++ was making inroads in industry it was also really too complex for teaching beginning programmers. Time and again I had lunch with other faculty in the department, and we all wondered "why isn't there a simpler version of C++?"

During one of these lunches, (APU had a meal plan to encourage faculty eating with the students, which I highly recommend), one of my colleagues, Dr. Bill McCarty started telling me about a simplified version of C++ being developed by a team at Sun Microsystems, named OAK. Through his contacts we were able to get some pre-release versions of the software, and through the Spring we learned the language and created applications we could use in our classes. Over the summer, we team-taught a standing-room only graduate course using the newly-released beta software, now renamed Java.

As a result of teaching this class, we received an offer from a specialty publisher, Waite Group Press, to write a book applying Java to the first year Computer Science course (CS1). We spent a year writing the book, Object Oriented Programming in Java, which was published in December of 1996. At the same time, we switched our introductory classes from C++ to Java, using drafts of our new textbook for the class. Both of these moves were more successful than our earlier introduction of C++ as the introductory programming language. The book received good reviews, such as this one at CNN:



<http://www.cnn.com/TECH/computing/9901/22/javabks.idg/index.html>

Other colleges and universities also began adopting Java (although I don't think that they were necessarily following our lead). Today, almost 70% of Computer Science programs use Java as the introductory programming language.

Today, I find myself in a similar, situation, but with quite a few differences. In 2002 Don Reuter and I introduced a certificate program designed to teach the new Microsoft .NET platform, including the Visual Basic .NET language, which Don taught, and the C# class that I taught. These two classes provide the foundation and background for students who want to earn their Microsoft Certified Application Developer certification. In that sense, these classes serve the same function as the Java Programming I class does for those students going on for a 4-year CS degree.

Our problem, from the beginning, was the complete lack of appropriate educational materials, not merely textbooks, but exercises, exams and other ancillary materials. If you look at any textbook catalog, you'll see that there are 40 or 50 Java textbooks designed for first-year programming students, but only a handful of VB.NET books. For C# (my class) the situation is even worse. I've changed books four times since 2002 and I'm pretty unhappy with the one I have now. I'm sure that I could do a better job. I have pretty firm ideas on how the topic should be taught, and I've had some experience writing such a book. However, I'm *not* proposing researching or writing such a book as part of my sabbatical proposal for several reasons.

First, I don't think that the traditional textbook publishing model is well-adapted to small specialized markets, markets that change frequently. Publishers of two of the textbooks I used, (and which students liked), let them go out of print when the software changed. (Actually, I guess the publishers are *well*-adapted; they respond to small markets by selling fewer titles.) In 1996 anyone, including Homer Simpson, could become an Internet guru, and the computer book publishing market (including textbooks) exploded. Today, with consolidation and mergers, there are really only two or three publishers in our field and they are only going after the most lucrative (read large) markets.

Second, I think that in general, moving away from corporate, "big-co" publishers is a growing movement in education. I think that more and more, we, as faculty, will be producing our own educational materials. And, I'm not sure that those materials will necessarily be traditional textbooks (even if self-published). The growth of the Internet has lead to a huge increase in user-generated content (blogs, Wikis, and social networks like My Space and Facebook), and I see that movement overflowing into education as well.

Which leads (*finally*, I know) to my third point. If more course content is going to be instructor and student generated, as opposed to publisher supplied, how will that content get created, how will the creators get paid (if at all), and what will the content look like? I believe (at least this week) that much of this content will be video and multimedia rather than traditional textbooks or even printed Web pages. My hope is that this content will be created by both instructors and by students, and it will be freely available and shared.

For those cynical skeptics among you, who think I'm a wide-eyed naïf, let me give you an example of what I'm talking about. For the past two years, I've had an additional responsibility here at OCC working with Pam Barvarz as one of the faculty coordinators of the Online Learning Program. This has been a marvelous experience for us both, because we've been able to meet and talk with so many of you from different disciplines. While doing some research for OLP, I came across the concept of Math-casts: teacher and student-created Web videos for teaching mathematics. Here's a page you can look at to read more about this; maybe you'll find it as exciting as I do.

<http://www.coolschooltools.com/wbm3/ebook/sn3/index.html>

For my sabbatical project, I want to research this topic and learn to create similar multimedia-based "learning objects" using my C# class as a test-bed. If I'm successful, I hope to return with techniques that other instructors can apply to their own discipline and classes.

Methods of study and research:

Since I am requesting a "split" sabbatical, my study and research plan has two pieces:

During the Fall 2007 semester I will:

1. Evaluate different "screen-casting" and course-creation software.
 - a. I already have some experience with Articulate Presenter and Camtasia studio. However Articulate Presenter is very expensive and I really haven't had the

opportunity to learn what Camtasia can do. This sabbatical will give me the opportunity to really learn the product in depth.

- b. I also want to do a thorough Internet/literature/review search with an eye towards evaluating and adopting open-source screen-casting and multimedia course-creation software. I feel that our goals and those of the open-source community are especially compatible, but successfully deploying such software requires a little larger time investment.
- 2. As a pilot project, I will create a scripted storyboard for a set of screen-casts.
 - a. The storyboard would be specifically tailored towards the needs of the C# .NET programming class (and could later be adapted for the VB.NET class)
 - b. We would use the storyboard as the "textbook" for CS 140 during the Spring semester after the first part of my sabbatical.

When I return from the sabbatical I envision creating, and having my students create, the screen-casts that will eventually replace the written textbook.

During the Spring 2009 semester I will focus on adding multimedia assessment to the mix. While there are quizzing tools, like those found in the Articulate QuizMaker package, or those that come as part of Camtasia, those don't really meet my needs. I believe that I can create software that does, but I'll need to learn how to use Macromedia Flash. Here's my plan for Spring 2009:

- 1. Learn the Adobe Flash development environment and the ActionScript programming language well enough to develop interactive "learning objects".
 - a. To do this, I'll start by taking a class (or two) on Flash. I'll probably take online-self-paced classes, just so I can get going faster.
 - b. I'll then teach myself how to use Adobe's "e-learning" assessment framework. This is a framework that makes creating assessments easier by doing a lot of the repetitive work.
 - c. Finally, I'll teach myself how to implement such assessments as SCORM objects so they can easily be integrated into our new WebCT Vista installation.
- 2. Once I have a handle on Flash, I will begin developing a set of assessments and exercises to go along with the screen-cast "textbook" that should now be in use.

Expected Results :

As you know, the purpose of a sabbatical is, (according to the application),

...to encourage faculty members to pursue professional growth leading to the development of increased competence and the improvement of instruction and/or redirection in their discipline.

Here are the ways that I feel this sabbatical proposal meets those needs as well as the criteria specified in the Sabbatical Leave Application instructions:

1. Increased competence
 - a. This will improve my teaching by giving me several tools and techniques I would not otherwise have the time to learn.
 - b. This will make me a more effective member of the faculty, since I'll have skills and experiences I can share with our community.
 - c. This will make the C# class more effective, since I can tailor the instruction rather than following along with a pre-written textbook.
2. Improvement of Instructional Services
 - a. This will increase the quality of my classes by getting students more involved in their own learning and in teaching others.
 - b. Classes will improve because the delivery mode will more closely match their learning styles.
3. Exceptional Opportunity
 - a. Although I didn't mention it in the narrative portion of my application, one of the main reasons I've applied for Fall 2007 is because it is the last semester that Shin Karasuda will be with us at OCC. Shin teaches two of our core CS classes (CS 150 and CS 250) and when he leaves, I'll be taking over those classes. Taking the first portion of the sabbatical before he leaves will reduce to the minimum the disruption to our CS program.
4. Length of Service
 - a. I started at OCC full-time in 1997, so I've been here 10 years.

Thank you for your consideration of this sabbatical leave proposal. If there are any questions regarding this proposal, then please contact me and I will provide additional information or make any appropriate changes.

Stephen Gilbert
Computer Science Department
<http://csjava.occ.cccd.edu/~gilberts>
sgilbert@occ.cccd.edu
x21173

July 23, 2009

To: Academic Senate
Orange Coast College

From: Stephen Gilbert
Professor, Computer Science Department
Orange Coast College

Re: Sabbatical Leave Report

Summary of Work Completed and Results

I've just completed a two-semester, split independent study/research sabbatical leave with the goal of "developing new skills in producing multimedia learning objects and applying those skills to a pilot project in our department's Microsoft .NET Programming courses." Unfortunately, by the time that I started working on the sabbatical, our .NET courses had become much less important in our department's strategic plan. Rather than wasting my time, I refocused my development on our Java programming courses.

Fall 2007

During the Fall 2007 semester, I concentrated on evaluating different screen-casting software (more details below). To do this, I created several different versions of a "screen-cast" lesson on installing the Java Development Kit for my CS 170 class. As I completed each screen-cast, I compared both the development time as well as the finished results. At the completion of this first sabbatical semester, I completed a set of screen-cast storyboards, focused on active-learning techniques, and designed to replace the textbook in our CS 170 class.

My original proposal envisioned creating (and having my students create) the screen-casts that would eventually replace the written textbook. This didn't go exactly as planned. After working with the screen-casting software during the Fall, I came to the conclusion that having the students do the recording wasn't feasible. So, during the Spring and Summer, I completed the screen-casts myself, for use during Fall 2008.

The Fall class was taught as a 50% hybrid class, with about 2.5 hours of active multimedia screen-casts each week, in addition to a 2.5 hour on-campus lecture/lab. (The actual raw running time of the screen-casts was about 1.25 hours each week, but since each screen-cast required students to complete an exercise as they listened to the lecture, the minimum time for students to complete the material was about twice that.)

After using the materials for the Fall 2008 class, I was surprised to find that they were quite (quite, quite) unpopular with both the advanced students and with the slower students, who I had originally thought would really enjoy them. The reason that the students gave, was kind of eye-opening to me: according to the students, the videos just took too much time. This surprised me, since we were able to complete a similar amount of material in my on-campus-only class during the previous two semesters as I was

developing the screen-cast storyboard. Once the multimedia lessons were produced, though, the students really didn't want to sit through them. Instead, they almost universally wanted to know what assignments they had to complete, and then wanted to "browse" until they found what they wanted to know. This reliance on "just-in-time" learning seems to apply to my quicker and slower students, almost across the board. The active-learning lessons, in contrast, really forces students to listen to the lectures and complete the exercises in a methodical, sequential way. (And I thought that was a *good* thing! Go figure.)

Spring 2009

In October, 2008, I submitted a request to modify my original sabbatical proposal to the Sabbatical Leave Committee (attached). I spent November and December 2008 exploring Rice University's Connexions program, with the intention of creating a free online version of the textbook. By January, though, when the second half of my sabbatical began, I came to the conclusion that the Rice platform really was a little too limiting.

While I'm still interested in developing such a textbook, I returned to my original proposal for Spring 2009. My original plan called for "learning the Adobe Flash development environment and the ActionScript programming language well enough to develop interactive learning objects." To this end, I completed the Flash Programming class here at OCC and learned how to create SCORM modules to create assessments that integrate into Blackboard Vista. My SCORM objects, weren't too successful, though, and I turned to other tools to create the kind of rich, active-learning assessments that really make a difference in learning something like programming.

As a result, I was able to create a set of interactive Java assessments, using Virginia Tech's Web-based Center for Automated Testing (Web-CAT). In addition to the Web-CAT assessments, I've been working on adding interactive assessments to the online portion of my class, using a custom Java testing framework that I've developed, along with an open-source Java interpreter. While I've been able to complete the framework itself, I'm still working on a way to automatically monitor the student's progress and gather the results as they make their way through the course. Originally, I'd planned to do this with SCORM inside Blackboard Vista. While I was able to create several SCORM modules, I wasn't able to get the behavior and integration into my content pages that I really wanted. Currently, I'm working on creating my own custom PHP application to automatically monitor my student's progress.

In general, I think that my sabbatical leave was successful, even though it took several twists and turns. I've certainly been able to accomplish my goal of "developing new skills in producing multimedia learning objects and applying those skills to a pilot project". More importantly, the sabbatical leave allowed me to experiment with different learning techniques, and explore different technologies that I simply wouldn't have had the time for, during the regular semester. Thank you for that opportunity.

-- Stephen Gilbert

Sabbatical Methods and Detailed Results

I think that the most useful way for me to provide a detailed report about what I did on my sabbatical is to just annotate my original sabbatical request document, focusing on my original plan and what I did to complete the plan.

During the Fall 2007 semester I will:

3. Evaluate different "screen-casting" and course-creation software.
 - a. I already have some experience with Articulate Presenter and Camtasia studio. However Articulate Presenter is very expensive and I really haven't had the opportunity to learn what Camtasia can do. This sabbatical will give me the opportunity to really learn the product in depth.

To learn about both Articulate Presenter and Camtasia Studio in depth, I first acquired copies of the latest versions of each program. The online learning program was able to provide me with a copy of Articulate Presenter version 5 (my own copy was an earlier version.) The Business and Computing Division provided me with a copy of Camtasia Studio version 4 (my personal copy of Camtasia was also earlier.) As I learned the program, Camtasia went through two more revisions; instead of purchasing individual copies of these, I downloaded the 30-day evaluation versions, and used those to make my comparison screen-casts.

I've placed the resulting presentations on the Computer Science Web server so that the committee members can examine them:

- **Articulate Presenter 4:** <http://csjava.occ.cccd.edu/~gilberts/java6JDK/>
 - Pro: created the smallest files (about 1/5 the size of the others)
 - Pro: easiest to create and (in my opinion) best looking.
 - Pro: best sound by far using the same recording equipment
 - Pro: text of recording is automatically available to students if notes were used in PowerPoint for the storyboard. (ADA compliance).
 - Pro: automatically create Word (and from that, PDF) storyboard files from PowerPoint slides along with narration for students who want to "read along" instead of watch the presentation.
 - Con: cannot record complex screen interactions like you can with Camtasia or other screen-casting software.
 - Con: quite expensive and company is not that responsive.
 - Con: all presentations had to be re-published when Flash Player 10 came out. Quite a hassle.

- **Camtasia Studio 3:** <http://csjava.occ.cccd.edu/~gilberts/JDK6-install/>
 - Pro: very good for recording live, impromptu screen interactions. I use Camtasia regularly for creating screen-cast solutions for each homework assignment.
 - Pro: multiple formats for output, including Flash and Quicktime.
 - Pro: inexpensive (around \$200, with upgrades around \$90).
 - Con: sound not as good as Articulate
 - Con: file sizes are much-much larger than Articulate when recording PowerPoint.
 - Con: has ability to do closed captions, but it is a separate and time-consuming manual process. (This example was close captioned.) While great for unrehearsed and unscripted screen-cast videos, making them ADA compliant requires you to listen to the entire video, create a transcript, and then use Camtasia to caption your screen-recording. This process took me several hours when I already had the narration scripted; it would be prohibitive without scripting.
- **Camtasia Studio 5:** <http://csjava.occ.cccd.edu/~gilberts/JDK6/>
 - Pro: more output options including the ability to resize the video for small screens (Pod-cast and You-Tube style).
 - Pro: better quality output for both video and sound.

Conclusion

Even though Articulate Presenter is expensive, Camtasia doesn't really match the results when it comes to formal, scripted lecture materials of the kind that are normally created in PowerPoint. On the other hand, Camtasia offers the ability to capture and narrate actual screen interactions, which is compelling if students are attempting a complex task.

As a result, for my pilot project, I decided to use Articulate for the main multimedia lectures, and embed very short Camtasia videos for homework assignments and "how-to" lessons. (Although Articulate allows you to embed Flash videos inside the Camtasia lessons, I didn't find the results satisfactory.)

- b. I also want to do a thorough Internet/literature/review search with an eye towards evaluating and adopting open-source screen-casting and multimedia course-creation software. I feel that our goals and those of the open-source community are especially compatible, but successfully deploying such software requires a little larger time investment.

One of the problems that I had both with Camtasia and Articulate Presenter is that both of them were "Windows only", while I was doing more and more work on my Mac, in my office. With Articulate, I could at least shoot screenshots on the Mac and incorporate those into my PowerPoint before producing the finished video on

Windows. In fact, I could create the whole presentation using PowerPoint for the Mac and then just do the recording and final production on the Mac.

Another problem was that my original project called for the students to make their own screen-casts, but I really wasn't in a position to ask them to purchase such expensive software; our department had no budget to put either program in the Computing Center either.

To address both of these problems, I looked for free and/or open-source screen-casting software that worked on both Windows and the Mac. I found and tried two different products.

- **ScreenToaster:** (long URL must all be in one line if copy and pasting)
http://www.screentoaster.com/watch/stVUjXQ0NIR11XQ1RcX1JeUFZR/javabat_cigar_party#
 - Pro: the only complete multi-platform solution (Linux, Windows, Mac)
 - Pro: automatically installs using a Java applet. (although you have to click OK to trust it, so that is a potential security risk, just like installing a program on your computer.)
 - Pro: Completely free, but you have to register, so I don't know if there's actually a catch down the road.
 - Pro: free hosting for your videos is included when you register at <http://www.screentoaster.com>
 - Con: sound is not that good
 - **Con: advertised features don't really seem to work.** Uploads to You Tube had an authentication error. Downloads in Flash format don't keep the sound. The only good results are by uploading to the ScreenToaster Web site, which makes me a little nervous. According to their site, they are a French startup, but the company seems a little "sketchy". I wouldn't really consider this if it wasn't the only product to work in Linux, the Mac and Windows. Since it doesn't work that well, though, I guess that point is moot.
- **Jing:** A free, (but not open source) program and service from Techsmith
<http://faculty.orangecoastcollege.edu/sgilbert/movies/eclipse-mac01.swf>
 - Pro: Jing works on the Mac and on Windows (but not Linux)
 - Pro: Jing is free and you get 2GB of hosting space on the Screencast.com Web service. (For \$15 yearly you can get Jing PRO that gives you slightly greater quality, allows you to use your Web-cam on Mac Leopard, and leaves your video unbranded.)
 - Pro: Jing can do screen-captures (like a stripped-down Snag-it) and short (5 minutes or less) screen-casts
 - Pro: Jing allows you to save your captures on your local machine

- o Pro: Quality is good and the company (Techsmith) is reliable and seems trustworthy.
- o Pro: doesn't require you to use their upload site (although you can).
- o Con: limited to 5 minutes (maybe this is an advantage, though. Forces you to create focused videos.)
- o Con: videos have the Jing brand on them. You can pay \$15 yearly for Jing Pro and get them "unbranded".
- o Con: videos are not the highest quality. You can pay \$15 yearly for Jing Pro and get them unbranded.

Conclusion

Jing meets my needs for a screen-casting solution. It allows me to create short screen-casts on my Mac (unlike Camtasia and Articulate which only work on Windows) and, since it's free, it allows me to have the students use it for screen-casts and short videos.

Update: during the Fall 2008 semester, along with my pilot project, I had my students use the screen-capture portion of Jing to help me evaluate their assignments and to provide some accountability that they were actually working on the online materials.

In addition to these two, I explored several other screen-casting solutions, but none of them really met my needs. In addition, my work with Camtasia convinced me that making screen-videos ADA compliant (with both transcripts for the deaf and audio for the blind) was really tedious. By the time I returned to school and began work on part 2, I was convinced that the only practical screen-casting solution still centered around Articulate Presenter, with Camtasia or Jing videos and screen-captures added as an adjunct to the main cast.

4. As a pilot project, I will create a scripted storyboard for a set of screen-casts.
 - a. The storyboard would be specifically tailored towards the needs of the C# .NET programming class (and could later be adapted for the VB.NET class)
 - b. We would use the storyboard as the "textbook" for CS 140 during the Spring semester after the first part of my sabbatical.

When I return from the sabbatical I envision creating, and having my students create, the screen-casts that will eventually replace the written textbook.

As I mentioned in the summary, when I came back from the first part of my sabbatical our .NET offerings had become much less important to our department. Instead of putting a lot of effort into creating materials that would never be reused, I decided to focus on Java instead of C#, and created the storyboards for the CS 170 class, where demand was increasing. Then, during the Spring and Summer, I recorded screen-casts for each of these storyboards, using Articulate Presenter.

Rather than including copies of each of these storyboards in this report, I've posted them on my OCC Web site, along with the corresponding active-learning screen-cast. Here are the URLs for each of those resources.

Storyboards and Screen-casts

- 📄 Storyboards are in PDF format
- 🎥 Screen-casts are Flash format

Historic Programming: Learn about machine language, assembly language, and high-level languages.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w01/CS170F0802A.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J002/player.html> (22 min.)

What about Java? Learn a little about the different versions of Java, and why Java is important for businesses and for Computer Science students.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w01/CS170F0802B.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J003/player.html> (14 min.)

Installing the JDK: Follow these instructions to install the Java Development Kit on your own computer so you can work at home.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w01/CS170F0802C.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J004/player.html> (14 min.)

"In-Class" Lab Prep: Learn how to prepare the weekly "in-class" Lab exercises. Covers formatting, screenshots, and PDF conversion.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w02/CS170F0803A.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J008/player.html> (9 min)

Installing a Text Editor: Learn to install and configure your programmers' code editor.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w02/CS170F0803B.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J009/player.html> (5 min.)

Java Mechanics: Learn how to edit, compile, and run Java programs.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w02/CS170F0802D.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J005/player.html> (27 min.)

Java Basics: Learn about the basic structure and grammar used to write Java programs.

- 📄 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w02/CS170F0802E.pdf>
- 🎥 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J006/player.html> (28 min)

Introducing Web-Cat: Learn how to use the online submission and grading program.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w02/CS170F0802F.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J007/player.html> (7 min)

Object Variables: Learn how to create objects, object variables and references.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w03/CS170F0804A.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J010/player.html> (9 min)

Actual Parameters: Learn to customize objects by providing each of them different initial values.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w03/CS170F0804B.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J011/player.html> (14 min)

Objects and Methods: Learn how to control the objects you create by using accessor and mutator methods.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w03/CS170F0804C.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J012/player.html> (13 min)

The JavaDocs: Learn how to read and interpret the Java API documentation.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w03/CS170F0804D.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J013/player.html> (16 min)

Values and References: Gain an understanding of how programs store data.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w03/CS170F0804E.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J014/player.html> (7 min)

Working Objects: Introducing Karel J. Robot and his World.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w03/CS170F0804F.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J015/player.html> (16 min)

Testing Karel: Learn how to use JUnit and Checkstyle to test your Java programs.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w04/CS170F0806A.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J016/player.html> (22 min)

Defining More Classes: Review class definitions, and work independently to create and test the PongScore class.

-  <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w04/CS170F0806B.pdf>
-  <http://faculty.orangecoastcollege.edu/sgilbert/movies/J017/player.html> (17 min)

Web-Cat & BlueJ: Learn how to configure and use BlueJ's submitter application to submit your multi-file projects for grading.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w04/CS170F0806C.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J018/player.html> (10 min)

Introducing Java Applets: Our first "graphics interlude" will show you how to write an applet in BlueJ and how to draw on the screen.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w04/CS170F0806D.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J019/player.html> (24 min)

Real Numbers: Learn how to use Java's real-number types.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w05/CS170F0808B.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J020/player.html> (29 min)

Expressions: Learn to use operators and operands to perform calculations.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w05/CS170F0808C.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J021/player.html> (19 min)

Side-Effects: Learn to use Java operators that modify their operands.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w05/CS170F0808D.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J022/player.html> (16 min)

Math and Characters: Learn to use the Math class and meet Java's char primitive type.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w05/CS170F0808E.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J023/player.html> (22 min)

More on Strings: Learn to use the methods available to String objects.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w05/CS170F0808F.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J024/player.html> (19 min)

Practice Midterm: Follow these instructions to take a practice programming exam and get some tips on how to succeed.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w06/CS170F0809B.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J025/player.html> (27 min)

Local Color: Learn how to create and use Color objects.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w07/CS170F0810D.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J046/player.html> (10 min)

Variables & Fonts: Make the Artist applet more flexible by using variables, measuring your window and using the Font class.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w07/CS170F0810C.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J026/player.html> (19 min)

O-O Graphics: Use the object-oriented ACM graphics classes to write a program that displays playing cards. 25 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w07/CS170F0810A.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J027/player.html>

True or False: Meet the boolean primitive type, learn how to respond to mouse events and to make simple decisions. 28 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w07/CS170F0810B.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J028/player.html>

Multiple Choice: Use sequential and nested if statements to code branches with multiple alternatives. 26 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w08/CS170F0812A.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J029/player.html>

The Switch: Efficient multi-way branches using Java's switch statement. 16 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w08/CS170F0812B.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J030/player.html>

Extending Karel: Increase Karel's intelligence by adding constructors, overriding and extending methods and using his predicate methods. 17 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w08/CS170F0812C.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J031/player.html>

Better Booleans: Expressions using NOT, AND, and OR, De Morgan's law. 11 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w08/CS170F0812D.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J032/player.html>

The while Loop: Learn to write counter-controlled and indefinite loops using Java's while statement. 26 min.

-  <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w09/CS170F0814A.pdf>
-  <http://faculty.orangecoastcollege.edu/sqilbert/movies/J033/player.html>

Jumps and Iterators: Learn how to use Java's do-while loop, control flow with the break and continue statements, and use Java's iterators to process text. 16 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w09/CS170F0814B.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J034/player.html>

Writing Loops: Step-by-step strategy for quickly writing correct loops. 20 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w09/CS170F0814C.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J035/player.html>

Eclipse@Home: Download, install and configure a professional Java IDE. 13 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w09/CS170F0814D.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J036/player.html>

Practice Midterm 2: Follow these instructions to take a practice programming exam and get some tips on how to succeed. 27 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w10/CS170F0815B.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J037/player.html>

Introducing Arrays: Create and initialize one-dimensional primitive arrays. 18 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w11/CS170F0817A.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J038/player.html>

Processing Arrays: Learn how to access array elements and how to write methods that process arrays. 12 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w11/CS170F0817B.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J039/player.html>

Arrays and Loops: Learn how to use Java's loop statements to perform common array-processing algorithms. 11 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w11/CS170F0817C.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J040/player.html>

Statics: Learn how to create and use class or static variables. Learn to write static methods. 9 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w12/CS170F0816B.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J041/player.html>

Scope: Recognize local and class scope in your Java programs. 9 min.

 <http://faculty.orangecoastcollege.edu/sqilbert/CS170-Online/w12/CS170F0816C.pdf>
 <http://faculty.orangecoastcollege.edu/sqilbert/movies/J042/player.html>

Overloading: Learn how to add overloaded methods to your classes and how to determine which method will be called. 10 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w12/CS170F0816D.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J043/player.html>

Swatting Bugs: Learn how to use the Eclipse debugger to suspend and examine your program as it runs. 30 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w12/CS170F0816A.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J044/player.html>

Unit Testing: Learn how to create unit tests using JUnit 4 and the Eclipse IDE. 24 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w12/CS170F0816E.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J045/player.html>

Classes and Arrays: Use encapsulation to hide the messy details of processing partially-filled arrays and resizing arrays when necessary. 15 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w13/CS170F0819A.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J047/player.html>

Generics: Learn how to use Java's generic ArrayList class to store any type of element. 20 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w13/CS170F0819B.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J048/player.html>

2-D Arrays: Learn how to create multi-dimensional arrays that can store and process rows and columns. 17 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w13/CS170F0819C.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J049/player.html>

Widgets: Learn how to add buttons and labels to your Java applications. 19 min.

 <http://faculty.orangecoastcollege.edu/sgilbert/CS170-Online/w13/CS170F0819D.pdf>
 <http://faculty.orangecoastcollege.edu/sgilbert/movies/J050/player.html>

Programming Final Sample Solution: After you've completed the programming sample exam, you can check this impromptu screen-cast to check your work. (36 min)

 <http://faculty.orangecoastcollege.edu/sgilbert/movies/SampleFinalSolution/index.html>

Spring 2009

During the Spring 2009 semester I will focus on adding multimedia assessment to the mix. While there are quizzing tools, like those found in the Articulate QuizMaker package, or those that come as part of Camtasia, those don't really meet my needs. I believe that I can create software that does, but I'll need to learn how to use Macromedia Flash. Here's my plan for Spring 2009:

3. Learn the Adobe Flash development environment and the ActionScript programming language well enough to develop interactive "learning objects".
 - a. To do this, I'll start by taking a class (or two) on Flash. I'll probably take online-self-paced classes, just so I can get going faster.
 - b. I'll then teach myself how to use Adobe's "e-learning" assessment framework. This is a framework that makes creating assessments easier by doing a lot of the repetitive work.
 - c. Finally, I'll teach myself how to implement such assessments as SCORM objects so they can easily be integrated into our new WebCT Vista installation.
4. Once I have a handle on Flash, I will begin developing a set of assessments and exercises to go along with the screen-cast "textbook" that should now be in use.

For the second half of my sabbatical, I focused on learning the Adobe Flash environment and the ActionScript 3 programming language. Instead of taking an online course, I purchased a copy of Adobe Flash Professional and then enrolled in the CIS 141 Flash Programming Class here at OCC (taught by Gabriella Ernsberger during Spring 2009).

I actually spent quite a bit of time on the class (more than two complete days each week), mostly due to learning a new language and framework, and, of course, my complete lack of artistic talent. In the end, I found myself pretty comfortable with ActionScript, but much less so with the Adobe Flash environment.

Here are two of the projects that I completed during the semester:

- ❶ Slide Show Program with buttons
<http://faculty.orangecoastcollege.edu/sgilbert/flash/slideshow/index.html>
- ❷ My rendering of the classic Space Invaders game (final project)
<http://faculty.orangecoastcollege.edu/sgilbert/flash/invaders/index.html>

Although I did learn Flash, I became convinced that it was not a sufficient platform to add the kinds of rich multimedia assessments that I wanted to use in my classes. There are several reasons for this:

- Since Adobe bought Macromedia Flash, they have discontinued the community supported "learning-interactions" framework and made it available only as part of the new CS4 Adobe E-Learning Suite (at about \$1,800). Since I already purchased

Adobe CS 3, I wasn't anxious to invest in something like this, just to find out that it wasn't what I needed.

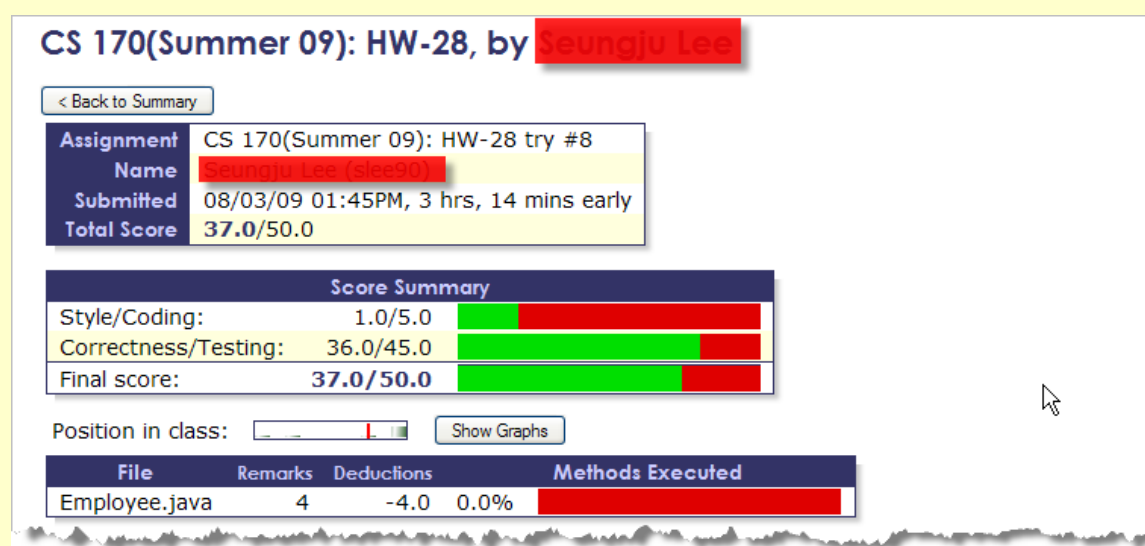
- Even though Flash is, well, "flashy", after learning ActionScript I discovered that it really didn't allow me to do what I wanted: allow the students to write short programs and get instant feedback on whether they are doing it right or not.
- The SCORM modules in Blackboard Vista don't really work very well. You have almost no control over how the assessment is presented or "played", and very little control over the reporting of grades to the LMS. In the end, I simply abandoned any attempt to create SCORM objects to work with BBV; I'm convinced that it is impossible to create the kind of seamless student experience that I desire.

I haven't abandoned my goals for creating rich and meaningful assessments, though. Instead, I've been exploring several different technologies for supplying those assessments (and capturing the results.) This is really a work in progress. Here's what I've done so far.

Web-CAT Assessments

With the help of Stephen Edwards at Virginia Tech, I've created a set of assignments that test whether students actually know how to program. I've used this same technique for both the practical midterm and programming exams. Using Edwards Web-CAT program (which Virginia Tech has graciously hosted for our CS 170 students), students complete their programs and then get immediate feedback on its correctness and programming style. This means that instead of submitting their assignments and waiting a week for me to return them, students can find out immediately if they've done the program correctly, and fix their errors as they work.

Here's a screenshot of the showing the kind of feedback report that the student receives:



Here's a screenshot showing the feedback for correctness errors, along with hints that the student receives when they submit their assignment.

Estimate of Problem Coverage (80%)

Problem coverage: 80%

For this assignment, the proportion of the problem that is covered by your solution is being assessed by running a suite of reference tests against your solution.

Test results indicate that your code still contains bugs. Your code appears to cover **only 80%** of the behavior required for this assignment.

Double check that you have carefully followed all initial conditions requested in the assignment in setting up your solution, and that you have also met all requirements for a complete solution in the final state of your program.

The following hint(s) may help you locate some ways in which your solution may be improved:

- public void Employee.raiseSalary(double, double) cannot be called with argument of type (Double): incorrect number of parameters

Employee.java

1 /**

And here's a screen-shot that shows what kind of style errors they've made in their programs:

```
66      * raise salary value.
67      * @param byPercent
68      * @param salary .
69      */
70 0 public void raiseSalary(double byPercent, double salary)

Error [Checkstyle]: -1
'salary' hides a field.

71
72      {
73 0          double raise = salary * byPercent / 100;
74 0          salary += raise;
75 0          this.salary = salary;
76      }
77
78      /**
79      * this is.
80      */
81
82 0 public String toString()
83 {
84
85 0     return "Name: " + name + "\nSalary: ";
86 }
87
88 }
89
```

< Back to Summary

I've created about fifty of these programming assessments. Its very time consuming creating the test suite for each assignment (about 4-8 hours, depending on the difficulty), but it saves me a considerable amount of time grading assignments. More importantly, though, is the feedback it provides the students and the improved learning environment it fosters.

JavaBat and JTester

In addition to the assessments I've created using Web-CAT, I've been using a suite of shorter, more focused online exercises developed by Professor Nick Parlante at Stanford University. Professor Parlante has posted these free for use on his Javabat Web site at <http://javabat.com>. I assign the exercises as homework assignments and the students can automatically submit them to my instructor account.

Here's a screenshot that shows what a typical Javabat exercise looks like:

JavaBat java practice problems [about](#) | [help](#) | [done](#) | [prefs](#) | [create account](#)

id/email
password

[forgot password](#)

Logic-1 > squirrelPlay

[prev](#) | [next](#) | [chance](#)

The squirrels in Palo Alto spend most of the day playing. In particular, they play if the temperature is between 60 and 90 (inclusive). Unless it is summer, then the upper limit is 100 instead of 90. Given an int temperature and a boolean isSummer, return true if the squirrels play and false otherwise.

squirrelPlay(70, false) → true
squirrelPlay(95, false) → false
squirrelPlay(95, true) → true

Students type code here

```
public boolean squirrelPlay(int temp, boolean isSummer) {  
  
}
```

As you can see, students can type in their code without installing any software on the machine they are using. That means they can complete these Javabat exercises at any computer they like, not just those in the lab. The software that tests their program lives on the Stanford servers, and, like Web-CAT, provides them with immediate feedback that looks something like this:

Alto spend most of the day playing. In particular, they play if the temperature is (inclusive). Unless it is summer, then the upper limit is 100 instead of 90. Given an int temperature and a boolean isSummer, return true if the squirrels play and false otherwise.

squirrelPlay(70, false) → true
squirrelPlay(95, false) → false
squirrelPlay(95, true) → true

Program correctness

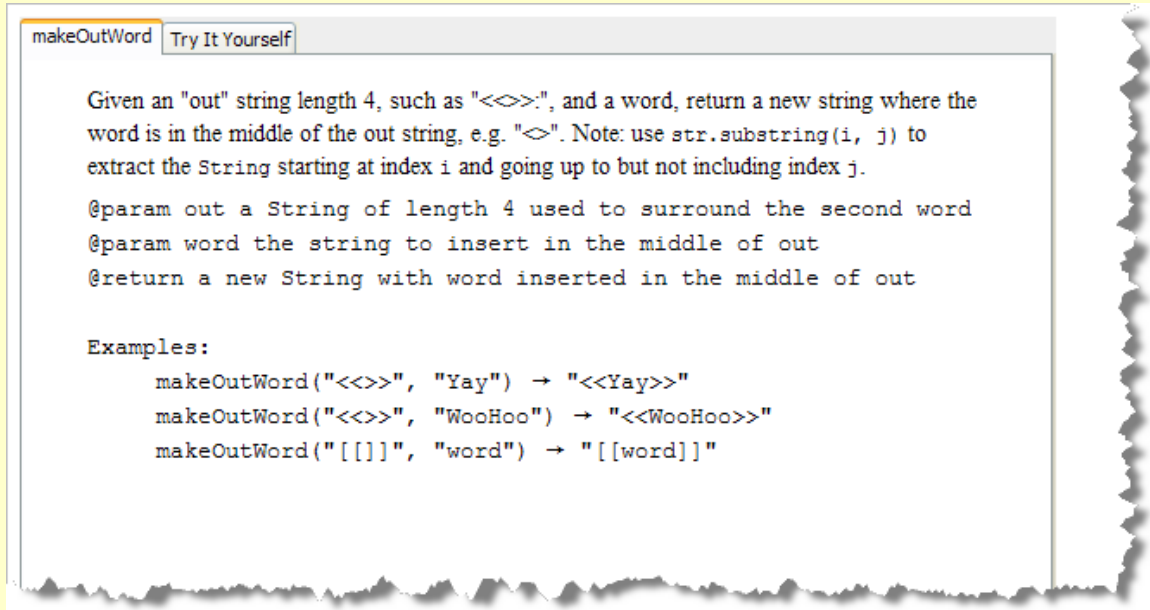
```
squirrelPlay(int temp, boolean isSummer) {
```

Expected	This Run	
squirrelPlay(70, false) → true	false	X
squirrelPlay(95, false) → false	false	OK
squirrelPlay(95, true) → true	false	X
squirrelPlay(90, false) → true	false	X
squirrelPlay(90, true) → true	false	X
squirrelPlay(50, false) → false	false	OK

Unlike the Web-CAT exercises, which are complete programs with extensive feedback, the exercises on the Javabat site are designed to focus on a single topic. While I like the Javabat exercises just fine, and there are hundreds of different exercises available, I'm not able to add

my own problems to the site, like I can with Web-CAT. Secondly, to do a Javabat exercise, students need to go to the Javabat Web site. That's fine for homework assignments, but it's not really ideal. What I want is to have interactive short exercises, like Javabat, with immediate feedback, that can be embedded directly into my online lessons. I have begun development on a Java applet based framework to do that, which I've named JTester.

With JTester, I'll can create an exercise like this, and embed it right inside my online lessons:



The screenshot shows a window titled "makeOutWord" with a tab labeled "Try It Yourself". The main text area contains the following instructions and examples:

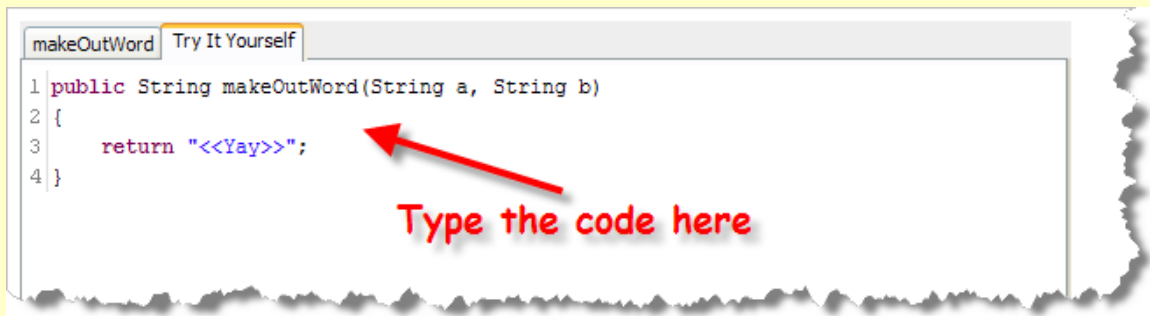
Given an "out" string length 4, such as "<<>>", and a word, return a new string where the word is in the middle of the out string, e.g. "<<>". Note: use `str.substring(i, j)` to extract the String starting at index `i` and going up to but not including index `j`.

@param out a String of length 4 used to surround the second word
@param word the string to insert in the middle of out
@return a new String with word inserted in the middle of out

Examples:

```
makeOutWord("<<>>", "Yay") → "<<Yay>>"
makeOutWord("<<>>", "WooHoo") → "<<WooHoo>>"
makeOutWord("[[]]", "word") → "[[word]]"
```

The student can click on Try It Yourself and type her code into the embedded text editor.

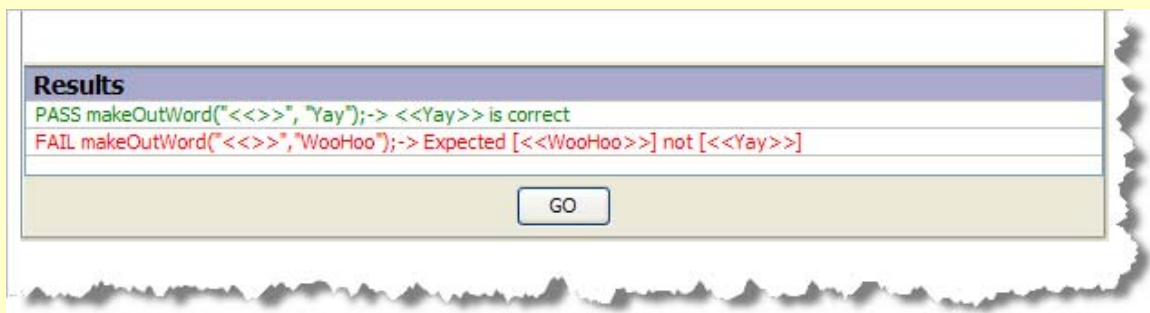


The screenshot shows the same window as before, but the code editor is now active. The code is:

```
1 public String makeOutWord(String a, String b)
2 {
3     return "<<Yay>>";
4 }
```

A red arrow points to the return statement, and the text "Type the code here" is written in red below the code editor.

When the student thinks she's correctly solved the problem, she can click on the GO button at the bottom of the editing window and get immediate feedback, just like Javabat.



The screenshot shows the results of the exercise. The window has a tab labeled "Results". The results are displayed in a table:

Results
PASS makeOutWord("<<>>", "Yay");-> <<Yay>> is correct
FAIL makeOutWord("<<>>", "WooHoo");-> Expected [<<WooHoo>>] not [<<Yay>>]

At the bottom of the window is a button labeled "GO".

I have completed the JTester prototype, which you can run (provided you have the Java plugin installed on your computer) at:

- <http://faculty.orangecoastcollege.edu/sgilbert/applets/JTester.html>

Currently (Summer 09) I'm working on several aspects of this program before I'll be able to deploy it with my online classes.

- I need to create a more flexible question format and question prep tool. I originally built a small GUI tool to enter questions and test cases, and then compile those down to a .class file format. I believe that the program will be more flexible if it can be made to work with applet parameters and an xml-based question format.
- Currently the program uses an embedded Java interpreter to test the student's code. I create some way for the students to submit their final results to my own server. I'd also like to keep the students results during the semester, so that they can see when they visit the page whether or not they have completed a particular exercise.